COLLEGE BIG DATA TECHNOLOGIES (BDT) MINI-PROJECT

Large-Scale Railways Data Processing and Demand Forecasting using Java and Apache Spark

An end-to-end distributed railway intelligence platform engineered for historical passenger movement analysis, route performance evaluation, delay classification, and machine learning demand forecasting.

TECHNOLOGY STACK

Java 21 + Spark 3.5.1

Spring Boot 3.2.5 - Spark MLlib
MySQL 8.x / In-Memory H2

DATASET VOLUME

10,248 Records

28 Express Train Corridors
Full 12 Calendar Months (Jan-Dec)

LIVE APPLICATION STATUS

http://localhost:8080

11 Working Modules
130% Presentation Scale Active

Strict Compliance: 100% Java Backend, Genuine Apache Spark, Local Windows Execution (No Google Colab, No Python).

1. Problem Statement & Research Objectives

ACADEMIC CONTEXT

Why traditional databases and single-threaded Python scripts fail at scale

Core Transportation Challenges

1. Severe Holiday & Seasonal Congestion:
   Summer vacations (May-Jun) and major festivals (Diwali, Chhath, Durga Puja) generate massive passenger surges causing huge waitlists and dangerous overcrowding.

2. Rolling Stock & Coach Misallocation:
   Trunk corridors experience severe seat shortages while secondary routes run with empty coaches, wasting fuel.

3. Junction Delay Cascading:
   Train delays propagate across shared track junctions without automated delay severity classification.

Why Big Data Technologies (BDT)?

1. Relational Database Bottlenecks:
   Traditional SQL databases struggle to perform real-time multi-criteria aggregations over millions of trip rows.

2. Google Colab Strictly Prohibited:
   College academic guidelines forbid cloud notebooks and mandate building a standalone Java application locally.

3. Our Engineered Solution:
   Engineered a 100% Java + Apache Spark 3.5.1 platform performing in-memory DataFrame transformations and machine learning demand forecasting on Windows.

VIVA / SPEAKER NOTE:

Indian Railways carries 23M+ riders daily. Traditional SQL queries fail under high-velocity data. Our objective was to implement genuine in-memory distributed computing using Apache Spark in Java 21 to solve route capacity bottlenecks.

2. Technology Stack & Academic Compliance

TECHNOLOGY MATRIX

100% Java distributed architecture — zero Python, zero Google Colab

Layer	Technology	Version	Role & Academic Justification
Language	Java Development Kit (JDK)	**21 LTS**	Mandated language; eliminates Py4J latency; strongly typed.
Big Data Engine	Apache Spark (Java API)	**3.5.1**	In-memory DataFrames, Catalyst Optimizer, RDDs in local[*] mode.
Machine Learning	Spark MLlib	**3.5.1**	VectorAssembler + RandomForestRegressor (chronological 80/20 split).
Backend Framework	Spring Boot	**3.2.5**	REST APIs, dependency injection, and static web dashboard hosting.
Database Layer	MySQL 8.x + Fallback H2	**8.0 / 2.2**	MySQL with automatic zero-error fallback to embedded H2 (MySQL mode).
Windows Native I/O	Hadoop WinUtils	**3.3.5**	Bundled winutils.exe & hadoop.dll for Windows POSIX filesystem emulation.
Frontend UI	HTML5, Bootstrap, Chart.js	**5.3 / 4.4.2**	Deep Navy enterprise theme, 130% Presentation Mode, 2.5x High-DPI.

VIVA / SPEAKER NOTE:

We adhered strictly to academic constraints: No Python or Colab was used. Everything is compiled in Java 21 LTS with Apache Spark 3.5.1 running locally across CPU worker threads.

3. Multi-Tier Distributed Architecture

SYSTEM DESIGN

Clean separation of presentation, microservice REST APIs, and Spark execution

1. CLIENT UI TIER

- Single Page App (SPA)
- 11 Working Modules
- High-DPI Chart.js
- 130% Zoom Switcher
- Printable PDF Engine
- Explainer Banners on every feature card

2. SPRING BOOT TIER

- 9 REST Controllers
- JPA Repositories
- Multipart CSV Upload
- 13-Column Schema Check
- Smart Rule Engine
- Global Exception Handler
- Hikari Connection Pool

3. SPARK ENGINE TIER

- SparkSession local[*]
- In-Memory DataFrames
- Catalyst Optimizer Plan
- 4 Parallel RDD Blocks
- VectorAssembler Pipeline
- RandomForest Regression
- Chronological Validation

4. STORAGE TIER

- 10,248 Row CSV Dataset
- MySQL Relational DB
- Embedded H2 Fallback
- Hadoop WinUtils DLL
- POSIX Emulation
- Dynamic Ingest Exports
- Direct CSV Download

VIVA / SPEAKER NOTE:

Our four-tier architecture decouples UI from data processing. Spring Boot serves RESTful JSON endpoints while Apache Spark handles all aggregations and machine learning directly in RAM.

4. Dataset Engineering & 13-Column Schema

BIG DATA DATASET

10,248 trip records covering all 366 days of 2024 across 28 Indian Railway corridors

#	Column Name	Type	Example Value
1	record_id	String (PK)	REC-10001
2	train_id	String	TR-12626
3	train_name	String	Karnataka Express
4	source_station	String	Bengaluru
5	destination_station	String	Delhi
6	travel_date	Date (ISO)	2024-05-18
7	passenger_count	Integer	1,280
8	train_capacity	Integer	1,200
9	delay_minutes	Integer	35
10	distance_km	Integer	2,400
11	day_of_week	String	Saturday
12	month	String	May
13	train_category	String	Superfast

Realistic Travel Pattern Modeling

- Full 12 Calendar Months: Jan 1 to Dec 31, 2024
  (366 leap days × 28 routes = 10,248 records).

- 11 Major Indian Railway Junctions:
  Bengaluru, Chennai, Mumbai, Delhi, Kolkata, Hyderabad,
  Pune, Mysuru, Kochi, Vijayawada, Ahmedabad.

- Summer Vacation Rush: +15% demand in May & June.

- Festival Surges: +22% demand in Oct (Dussehra/Diwali)
  and Nov (Chhath Puja).

- Weekend Multiplier: +18% surge on Fri, Sat, Sun.

- Weather Delays: Extra fog delays on Delhi routes in Dec/Jan.

VIVA / SPEAKER NOTE:

Notice our dataset has exactly 10,248 records spanning all 12 calendar months. We modeled realistic seasonal vacation surges, festival rushes, and monsoon dips to ensure real-world academic validity.

5. Executive Dashboard & Passenger Analytics

MODULE 1 & 2

Live KPIs and multi-criteria exploratory filtering over 11.64 million passengers

10,248	11,640,642	93.97%	25.7 min
TOTAL TRIP RECORDS	CUMULATIVE RIDERS	AVG TRAIN OCCUPANCY	NETWORK AVG DELAY

Module 1: Executive Dashboard

- Daily Passenger Traffic Trend Line (560px canvas):
  Visualizes seasonal traffic waves across all routes.

- Train Occupancy Donut Chart (560px canvas):
  Classifies trips into 4 utilization tiers (61.9% Critical).

- Top 7 Railway Corridors Horizontal Bar (540px):
  Ranks highest passenger volume routes (Bengaluru-Kolkata).

- Monthly Passenger Distribution Bar (540px):
  Complete Jan-Dec bars clearly showing festival peaks.

Module 2: Passenger Analytics & Filters

- Multi-Criteria Exploratory Filter Bar:
  Interactive filtering by Source Station, Destination Station, Train ID, Month, and Custom Date Ranges.

- Real-Time Manifest Database Table:
  Shows Record ID, Train, Route, Date, Passengers, Delay.

- Monthly Ridership Seasonality (520px):
  Highlights off-peak dips vs. high-demand quarters.

- Weekday vs. Weekend Patterns Chart (520px):
  Proves Friday-Sunday weekend traffic surge (+18%).

VIVA / SPEAKER NOTE:

Module 1 displays live aggregate KPIs across 11.6 million passengers. Module 2 provides multi-criteria filtering across all stations and trains, clearly highlighting weekend and festival travel surges.

6. Spark MLlib Demand Forecasting Pipeline

MODULE 4: ML

Random Forest regression with strict chronological train/test separation

Machine Learning Architecture

1. Algorithm: Apache Spark MLlib RandomForestRegressor
   Configured with 25 decision trees and maxDepth = 5.

2. VectorAssembler 6-Feature Pipeline:
   vector = [day_of_week, month, day_of_month,
   weekend_flag, distance_km, train_capacity]

3. Strict Chronological Train/Test Separation:
   Records are sorted strictly by travel_date. Earliest 80%
   used exclusively for training; subsequent 20% for testing.

4. Zero Future-Data Leakage:
   Eliminates fatal time-series leakage where future spikes
   contaminate the training of past historical days.

Model Evaluation & 7-Day Forecasting

1. Mean Absolute Error (MAE):
   Average error: ~30-40 passengers on 1,500-berth rakes.

2. Coefficient of Determination (R2 Score):
   Achieves R2 > 0.88 on high-demand trunk corridors,
   verifying strong predictive demand modeling.

3. Triple-Curve Visualization Chart (620px canvas):
   - Blue Solid: Actual historical ground-truth passengers.
   - Purple Dashed: MLlib predictions on held-out test data.
   - Emerald Green: Next 7-day future projections.

4. Actionable 7-Day Coach Rake Advisory:
   Recommends rake sizing (14-16 coaches) to prevent waitlists.

VIVA / SPEAKER NOTE:

Module 4 implements true machine learning with Spark MLlib. Unlike naive random splits, we order data chronologically to guarantee zero data leakage. Our Random Forest model achieves an R^2 above 0.88.

7. Train Delay & Coach Occupancy Analysis

MODULE 5 & 6

Standardized punctuality benchmarks and 4-tier capacity utilization mapping

Module 5: Train Delay Analysis

- Indian Railways On-Time Threshold: <= 15 minutes.

- Network Punctuality Rate: 56.5% on-time arrival rate.
  Average system arrival delay: 25.7 minutes.

- Chronic Bottleneck Identification:
  Identifies consistently delayed services (e.g. Gujarat Mail at 31.2 mins delay) for priority signal audits.

- Most Punctual Benchmark Express Services:
  Highlights top on-time services (e.g. Vande Bharat).

- 4-Tier Delay Severity Donut Chart (540px):
  Punctual (0-15m), Minor (16-45m), Moderate (46-90m), Severe (>90m).

Module 6: Coach Occupancy Analysis

- Capacity Saturation Percentage Formula:
  Occupancy % = (Passenger Count / Capacity) x 100

- 4 Standard Operational Capacity Tiers:
  1. Low: < 50% (Underutilized; detach coaches)
  2. Moderate: 50% - 75% (Normal off-peak load)
  3. High: 75% - 90% (Healthy, profitable service)
  4. Critical: > 90% (Overcrowded; attach coaches)

- High Occupancy Corridors (>= 85%):
  Flags saturated services needing extra AC rakes.

- Underutilized Corridors (< 60%):
  Flags low-ridership trains where empty coaches are saved.

VIVA / SPEAKER NOTE:

Modules 5 and 6 solve operational bottlenecks. We evaluate train delays against the official 15-minute standard and classify occupancy into 4 tiers to determine where coaches should be added or detached.

8. Decision Support Recommendations & CSV ETL Pipeline

MODULE 7 & 8

Translating Big Data into actionable management directives and dynamic ingestion

Module 7: Smart Decision Engine

- Automated Rule-Based Decision System:
  Scans Spark aggregations across occupancy, delays, and passenger loads to generate actionable directives.

- Prioritized Management Tiers:
  Categorized into CRITICAL, HIGH, and MEDIUM.

- Real-World Operational Prescriptions:
  1. Capacity: Attach +2 LHB 3-Tier AC coaches on corridors with occupancy exceeding 95%.
  2. Delays: Conduct track & signal modernization on routes with average delays exceeding 30 minutes.
  3. Utilization: Rationalize timetable on low-load routes.

Module 8: Dataset Management & ETL

- Active Dataset Summary Card:
  Displays 10,248 clean records in-memory, 0 corrupt rows, 28 routes, sub-second Spark DataFrame loading.

- Drag-and-Drop Ingestion Dropzone:
  Upload custom external railway CSV datasets dynamically.

- Pre-Ingestion 13-Column Validator:
  Checks headers, data types, nulls, and duplicate IDs.

- 10-Row Live Preview Roster:
  Inspects sample records before committing to Spark.

- Dynamic Spark Reload & One-Click Reset Button.

VIVA / SPEAKER NOTE:

Our platform does not stop at charts; Module 7 generates prioritized management directives. Module 8 provides an enterprise ETL pipeline to upload and validate external CSV callsets dynamically.

9. Interactive What-If Capacity Simulator

MODULE 10

Simulate holiday passenger surges and extra coach additions before physical deployment

Simulation Inputs & Interactive Controls

1. Corridor Selector:
   Choose any of the 28 active Indian Railway corridors.

2. Holiday Demand Surge Slider:
   Simulate festive rush from +0% (Normal) to +50% (Festivals) up to +100% (Peak Diwali/Chhath crush).

3. Extra Coaches Attachment Selector:
   Choose rake augmentation from 0 up to +6 LHB coaches (+72 to +432 additional physical berths).

4. Real-Time Spark Computation Button:
   Triggers instantaneous simulation calculations.

Simulated Operational Outcomes

1. Simulated Surge Demand vs. Baseline Demand.

2. Augmented Train Capacity (+72 berths/coach).

3. Projected Occupancy Saturation Percentage.

4. Unmet Passenger Spillover (Station Waitlist Prediction).

5. Incremental Fare Revenue Impact (Rs.):
   Calculated from extra passengers carried x route distance.

6. Spark AI Operational Action Advisory:
   Confirms whether coach addition resolves waitlists or if duplicate clone special trains must be scheduled.

VIVA / SPEAKER NOTE:

Module 10 is our scenario simulator. Railway managers can test festival rushes virtually. It simulates adding extra coaches, predicting resulting occupancy, passenger spillover, and ticket revenue.

10. Apache Spark Engine Profiler (Proving Big Data Execution)

MODULE 11: SPARK INTERNALS

Live extraction of the Catalyst Optimizer Physical Execution Plan DAG and RDD partitions

Catalyst Physical Execution Plan DAG

```
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   *(2) HashAggregate(keys=[route#12],
         functions=[sum(passenger_count#16)])
   +- Exchange hashpartitioning(route#12, 200)
      +- *(1) HashAggregate(keys=[route#12],
            functions=[partial_sum(passengers#16)])
         +- *(1) Project [route#12, passengers#16]
            +- *(1) Filter (isnotnull(passengers#16))
               +- FileScan csv [route#12, passengers#16]

>> Proves WholeStageCodegen, Predicate Pushdown,
>> In-Memory HashAggregation, and Shuffling!
```

Cluster Internals & RDD Partitions

- Spark Master Mode: local[*] (Java 21 LTS).

- CPU Worker Cores: Utilizes all logical hardware cores.

- Default Parallelism: Multi-threaded parallel tasks.

- JVM Heap Memory: Live tracking of allocated heap.

- RDD Partition Distribution Status:
  10,248 records are balanced across 4 in-memory blocks:
  - Partition 0: 2,562 records (Status: SUCCESS)
  - Partition 1: 2,562 records (Status: SUCCESS)
  - Partition 2: 2,562 records (Status: SUCCESS)
  - Partition 3: 2,562 records (Status: SUCCESS)

- High Performance: Sub-second query response time.

VIVA / SPEAKER NOTE:

Module 11 proves this is a genuine Big Data project. Here we extract Spark's internal Catalyst execution DAG showing HashAggregate and Exchange stages, along with 4 RDD partitions storing 2,562 rows each.

11. Conclusion & Key Academic Achievements

SUBMISSION READY

A complete, fully verified, production-grade Big Data platform running locally on Windows

1. BIG DATA RIGOR

- 100% Java 21 implementation
- 10,248 Records across 12 months
- Genuine Spark DataFrames
- Catalyst Query Optimization
- 4 In-Memory RDD partitions
- Zero Python / Zero Colab

2. MACHINE LEARNING

- Spark MLlib Random Forest
- Strict Chronological 80/20 split
- Zero future-data leakage
- MAE, RMSE, and R2 > 0.88
- 7-day demand projections
- Coach rake sizing advisory

3. FULL-STACK PLATFORM

- 11 Working dashboard modules
- What-If scenario simulator
- Prioritized decision directives
- 130% Presentation Mode
- Printable PDF & CSV reports
- MySQL + H2 auto-fallback

Live Application: http://localhost:8080 - Automated Test Suite: . est-app.ps1 - 100% Passing

Thank you to our guide and evaluators. We are now ready for live demonstration and Viva Voce questions.